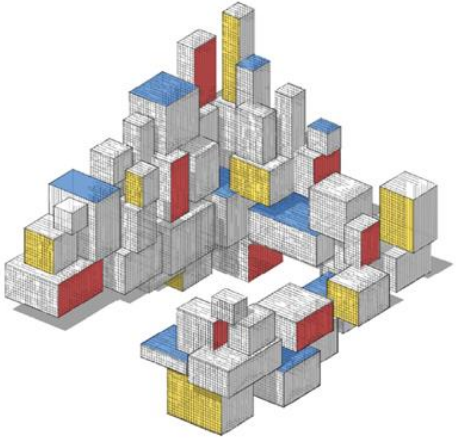
# Chapter 2 - Part 2
# Importing, Summarizing and Visualizing Data



Instructor: Dr. Heyfa Ammar

# Purpose

In this lecture we discuss:

- Where to find useful data sets
- How to load them into Python.
- How to (re)structure the data
- How to summarize data via tables and figures

descriptive, predictive, prescriptive

Readers unfamiliar with Python are advised to read the appendix of DSML first.

Python code and data sets for each chapter can be downloaded from the GitHub site: `https://github.com/DSML-book`

# Data

**Data can be thought of as being the result of some random experiment** — an experiment whose outcome cannot be determined in advance.

Theres at least some percentage of randomness in experiments
Ex: 2 houses with the same area and # of bedrooms arent always the same price (non-deterministic)

Data are often stored in a table or spreadsheet.

Convention: variables or features correspond to **columns** and the individual items (or units) correspond to rows.

features/attributes

units

Three types of columns:

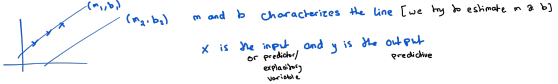
- One column is often an identifier or index column.
- Certain columns can correspond to the design of the experiment.
- Other columns represent the observed measurements of the experiment. Usually, these measurements exhibit **variability**.

What u assume or who u do the experiment on

ex 8 patients: Age > 65; Heart attacks (no other age grps could go bc)

ex 2: Interviewing CCIS students only since art or other majors might skew the results

# Data




Three types of columns:

- One column is often an identifier or index column.
- Certain columns can correspond to the design of the experiment.

  These record choices that were ***fixed or controlled by the experimenter*** before the data was collected. They describe the setup or conditions under which each observation was made, not an outcome that was measured.

  Example: In the nutrition_elderly example: `gender` or `situation` could be thought of as characteristics that define *which group* a person belongs to, rather than something being measured on them.
- Other columns represent the observed measurements of the experiment. Usually, these measurements exhibit **variability**.
  - Example: In the nutri data: `height`, `weight`, `tea`, `coffee`, `age` — these are measured values that vary across the 226 individuals

**Why the distinction matters:**

**Design variables** tell you *how the data was collected* or *what group something belongs to*: called *predictors* or *explanatory variables*

**Measurement variables** tell you *what was observed*: **what you're trying to explain, predict.**

# Data Repositories

A well-known repository of data sets is the Machine Learning Repository maintained by the University of California at Irvine (UCI):

https://archive.ics.uci.edu/

These data sets are typically stored in a CSV (comma separated values) format.

For example, to access the **abalone** data set from this website with Python, download the file to your working directory, import the **pandas** package via

```
import pandas as pd
```

and read in the data as follows:

```
abalone = pd.read_csv('abalone.data', header = None)
```

# Data Repositories

Another useful repository of over 1000 data sets from various packages in the R programming language, collected by Vincent Arel-Bundock, can be found at:

https://vincentarelbundock.github.io/Rdatasets/datasets.html

For example, to read Fisher's famous **iris** data set from R's **datasets** package into Python, type:

```
urlprefix = 'https://vincentarelbundock.github.io/Rdatasets/csv/'
dataname = 'datasets/iris.csv'
iris = pd.read_csv(urlprefix + dataname)
```

Concatenate 2 strings

# Iris Data

The iris data set contains four physical measurements (sepal/petal length/width) on 50 specimens (each) of 3 species of iris: setosa, versicolor, and virginica.

The output of **read_csv** is a **DataFrame** object, which is **pandas**'s implementation of a spreadsheet.

```
iris.head()
```

```
   Unnamed: 0  Sepal.Length  ...  Petal.Width  Species
0           1           5.1  ...          0.2   setosa
1           2           4.9  ...          0.2   setosa
2           3           4.7  ...          0.2   setosa
3           4           4.6  ...          0.2   setosa
4           5           5.0  ...          0.2   setosa

[5 rows x 6 columns]
```

The names of the features can be obtained via the **columns** attribute of the **DataFrame** object, as in **iris.columns**.

# Abalone Data

The first three rows of the abalone data set from the UCI repository can be found as follows:

```
abalone.head(3)
```

```
   0  1      2      3      4       5       6       7      8
0  M  0.455  0.365  0.095  0.5140  0.2245  0.1010  0.150  15
1  M  0.350  0.265  0.090  0.2255  0.0995  0.0485  0.070   7
2  F  0.530  0.420  0.135  0.6770  0.2565  0.1415  0.210   9
```

We can manually add the names of the features to the DataFrame by reassigning the **`columns`** attribute, as in:

```
abalone.columns = ['Sex', 'Length', 'Diameter', 'Height',
  'Whole weight', 'Shucked weight', 'Viscera weight',
  'Shell weight', 'Rings']
```

# Structuring Features According to Type

**Quantitative features** possess "numerical quantity", such as height, age, number of births, etc., and can either be **continuous** or **discrete**.

- Continuous quantitative features take values in a continuous range of possible values, such as height, voltage, or crop yield,
- Discrete quantitative features have a countable number of possibilities, such as a count.

**Qualitative features** do not have a numerical meaning, but their possible values can be divided into a fixed number of categories, such as {M,F} for gender or {blue, black, brown, green} for eye color. For this reason such features are also called **categorical** or **factors**.

# Nutrition Data

When manipulating, summarizing, and displaying data, it is important to correctly specify the type of the variables (features).

We illustrate this using the **nutrition_elderly** data set.

```
xls = 'http://www.biostatisticien.eu/springeR/nutrition_elderly.xls'
nutri = pd.read_excel(xls)
```

This creates a DataFrame object **nutri**.

```
pd.set_option('display.max_columns', 8) # to fit display
nutri.head(3)
```

```
   gender  situation  tea ...   cooked_fruit_veg  chocol  fat
0       2          1    0 ...                  4       5    6
1       2          1    1 ...                  5       1    4
2       2          1    0 ...                  2       5    4

[3 rows x 13 columns]
```

# Checking Variable Types

Check the type of the variables via the **info** method:

```
nutri.info()
```

Summary of all the features of the data set with the respective types

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 226 entries, 0 to 225
Data columns (total 13 columns):
gender              226 non-null int64
situation           226 non-null int64
tea                 226 non-null int64
coffee              226 non-null int64
height              226 non-null int64
weight              226 non-null int64
age                 226 non-null int64
meat                226 non-null int64
fish                226 non-null int64
raw_fruit           226 non-null int64
cooked_fruit_veg    226 non-null int64
chocol              226 non-null int64
fat                 226 non-null int64
dtypes: int64(13)
memory usage: 23.0 KB
```

Feature

total features

type of each feature

| gender | situation | tea | coffee | height | weight | age | meat | fish | raw_fruit | cooked_fruit_veg | chocol | fat |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 1 | 0 | 0 | 151 | 58 | 72 | 4 | 3 | 1 | 4 | 5 | 6 |
| 2 | 1 | 1 | 1 | 162 | 60 | 68 | 5 | 2 | 5 | 5 | 1 | 4 |
| 2 | 1 | 0 | 4 | 162 | 75 | 78 | 3 | 1 | 5 | 2 | 5 | 4 |
| 2 | 1 | 0 | 0 | 154 | 45 | 91 | 0 | 4 | 4 | 0 | 3 | 2 |
| 2 | 1 | 2 | 1 | 154 | 50 | 65 | 5 | 3 | 5 | 5 | 3 | 2 |
| 2 | 1 | 2 | 0 | 159 | 66 | 82 | 4 | 2 | 5 | 5 | 1 | 3 |
| 2 | 1 | 2 | 0 | 160 | 66 | 74 | 3 | 3 | 5 | 5 | 5 | 6 |
| 2 | 1 | 0 | 2 | 163 | 66 | 73 | 4 | 2 | 5 | 5 | 1 | 6 |
| 2 | 1 | 0 | 3 | 154 | 60 | 89 | 4 | 3 | 5 | 5 | 5 | 6 |
| 2 | 1 | 0 | 2 | 160 | 77 | 87 | 2 | 3 | 5 | 4 | 0 | 3 |
| 2 | 1 | 0 | 2 | 175 | 68 | 91 | 5 | 2 | 5 | 5 | 5 | 3 |
| 2 | 1 | 2 | 0 | 165 | 75 | 81 | 5 | 2 | 2 | 3 | 0 | 6 |
| 2 | 1 | 0 | 3 | 158 | 53 | 89 | 4 | 2 | 5 | 5 | 5 | 4 |
| 2 | 1 | 2 | 0 | 155 | 63 | 79 | 3 | 1 | 5 | 3 | 2 | 4 |
| 2 | 1 | 0 | 2 | 154 | 80 | 83 | 3 | 3 | 5 | 5 | 1 | 3 |
| 1 | 1 | 0 | 3 | 166 | 80 | 78 | 5 | 0 | 5 | 3 | 0 | 4 |
| 2 | 1 | 0 | 2 | 159 | 57 | 74 | 3 | 2 | 5 | 3 | 1 | 4 |
| 2 | 1 | 0 | 2 | 157 | 55 | 74 | 3 | 2 | 5 | 5 | 2 | 3 |
| 1 | 1 | 0 | 1 | 165 | 57 | 78 | 5 | 2 | 5 | 5 | 5 | 4 |
| 2 | 1 | 0 | 0 | 156 | 90 | 78 | 5 | 1 | 5 | 5 | 2 | 4 |
| 1 | 2 | 0 | 1 | 175 | 90 | 73 | 5 | 1 | 5 | 3 | 2 | 8 |
| 2 | 2 | 2 | 2 | 161 | 68 | 76 | 4 | 2 | 5 | 5 | 1 | 1 |
| 1 | 2 | 2 | 2 | 168 | 83 | 85 | 4 | 2 | 5 | 5 | 1 | 1 |
| 1 | 1 | 0 | 4 | 168 | 90 | 73 | 3 | 2 | 4 | 5 | 0 | 3 |
| 2 | 2 | 3 | 1 | 156 | 56 | 77 | 3 | 2 | 5 | 3 | 0 | 4 |
| 1 | 2 | 3 | 2 | 170 | 70 | 74 | 3 | 2 | 5 | 3 | 0 | 4 |
| 2 | 1 | 0 | 0 | 162 | 58 | 67 | 4 | 3 | 5 | 5 | 0 | 3 |
| 1 | 2 | 4 | 5 | 172 | 80 | 68 | 5 | 3 | 5 | 5 | 5 | 5 |
| 2 | 1 | 1 | 1 | 165 | 75 | 69 | 4 | 3 | 5 | 5 | 5 | 3 |
| 1 | 2 | 0 | 2 | 171 | 81 | 71 | 4 | 2 | 5 | 5 | 5 | 3 |
| 2 | 2 | 0 | 1 | 160 | 54 | 67 | 4 | 4 | 5 | 3 | 2 | 4 |

How do you interpret the highlighted values ?

We need to convert to category to understand

Convert qualitative(numbers) to qualitative data

# Classifying the Features in nutri

All 13 features in **nutri** are (at the moment) interpreted by Python as quantitative variables, indeed as integers, simply because they have been entered as whole numbers.
The following table shows how the variable types should be classified.

| | |
|---|---|
| Qualitative | gender, situation, fat |
| | meat, fish, raw_fruit, cooked_fruit_veg, chocol |
| Discrete quantitative | tea, coffee |
| Continuous quantitative | height, weight, age |

Table: Description of the variables in the nutritional study.

| Feature | Description | Unit or Coding |
|---|---|---|
| gender | Gender | 1=Male; 2=Female |
| situation | Family status | 1=Single<br>2=Living with spouse<br>3=Living with family<br>4=Living with someone else |
| tea | Daily consumption of tea | Number of cups |
| coffee | Daily consumption of coffee | Number of cups |
| height | Height | cm |
| weight | Weight (actually: mass) | kg |
| age | Age at date of interview | Years |
| meat | Consumption of meat | 0=Never<br>1=Less than once a week<br>2=Once a week<br>3=2–3 times a week<br>4=4–6 times a week<br>5=Every day |
| fish | Consumption of fish | As in meat |
| raw_fruit | Consumption of raw fruits | As in meat |
| cooked_fruit_veg | Consumption of cooked fruits and vegetables | As in meat |
| chocol | Consumption of chocolate | As in meat |
| fat | Type of fat used for cooking | 1=Butter<br>2=Margarine<br>3=Peanut oil<br>4=Sunflower oil<br>5=Olive oil<br>6=Mix of vegetable oils (e.g., Isio4)<br>7=Colza oil<br>8=Duck or goose fat |

1, 2, 3, 4 isn't understandable
but single, living with spouce is understandable

# Recoding Categorical Features

We can modify the Python value and type for each categorical feature, using the **replace** and **astype** methods. For categorical features, such as **gender**, we can replace the value 1 with **'Male'** and 2 with **'Female'**, and change the type to **'category'** as follows.

```
DICT = {1:'Male', 2:'Female'}  # dictionary specifies replacement
nutri['gender'] = nutri['gender'].replace(DICT).astype('category')
```

change 1 as male
2 as female

The structure of the other categorical-type features can be changed in a similar way. Continuous features such as **height** should have type float:

```
nutri['height'] = nutri['height'].astype(float)
# Repeat this for the other variables.
```

change the height to float

weight to float, etc.

# Summary Tables

It is often useful to summarize a large spreadsheet of data in a more condensed form.

A table of counts or a table of frequencies makes it easier to gain insight into the underlying distribution of a variable, especially if the data are qualitative.

Such tables can be obtained with the methods `describe` and `value_counts`.

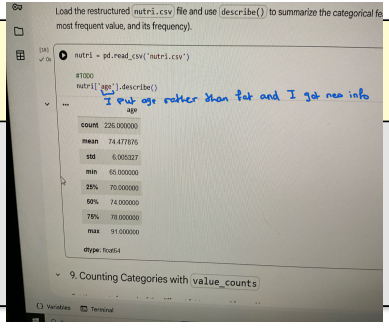
# Summarizing a Categorical Feature

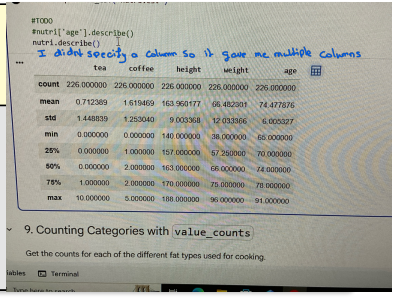
As an example, we load the `nutri` DataFrame, which we restructured and saved as `'nutri.csv'`, and then construct a summary for the feature (column) `'fat'`.

```
nutri = pd.read_csv('nutri.csv')
nutri['fat'].describe()
```

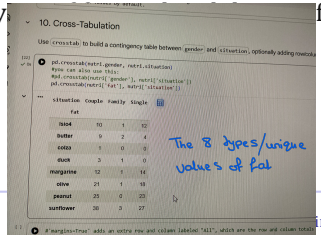
Changing this to other cols gives u new info

```
count        226
unique         8   # 8 unique values of fat
top     sunflower  # mostly included
freq          68  # how many times it was included
Name: fat, dtype: object
```




We see that there are 8 different ty flower has the highest count, with 68 out of 226 individuals using this type of cooking fat.



# value_counts

The method `value_counts` gives the counts for the different fat types.

```
nutri['fat'].value_counts()
```

```
sunflower    68
peanut       48
olive        40
margarine    27
Isio4        23
butter       15
duck          4
colza         1
Name: fat, dtype: int64
```

Column labels are also attributes of a DataFrame, and nutri.fat, for example, is exactly the same object as nutri['fat'].

# Cross-Tabulation

It is also possible to use **crosstab** to cross tabulate between two or more variables, giving a contingency table:

```
pd.crosstab(nutri.gender, nutri.situation)
```

```
situation  Couple  Family  Single
gender
Female         56       7      78
Male           63       2      20
```

We see, for example, that the proportion of single men is substantially smaller than the proportion of single women. To add row and column totals, use **margins=True**.

```
pd.crosstab(nutri.gender, nutri.situation, margins=True)
```

```
situation  Couple  Family  Single  All
gender
Female         56       7      78  141
Male           63       2      20   85
All           119       9      98  226
```

# Summary Statistics: Mean

In the following, $x = [x_1,...,x_n]^T$ is a column vector of n numbers. For our nutri data, the vector x could, for example, correspond to the heights of the n = 226 individuals.

The sample mean of x, denoted $\bar{x}$, is simply the average of the data values:

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

Using the `mean` method in Python for the nutri data, we have, for instance:

```
nutri['height'].mean()
```

```
163.96017699115043
```

# Quantiles and Percentiles

The p-**sample quantile** ($0 < p < 1$) of x is a value x such that at least a fraction p of the data is less than or equal to x and at least a fraction $1 - p$ of the data is greater than or equal to x.

The **sample median** is the sample 0.5-quantile. The p-sample quantile is also called the $100 \times p$ percentile. The 25, 50, and 75 sample percentiles are called the first, second, and third **quartiles** of the data.

For the nutri data they are obtained as follows.

```
nutri['height'].quantile(q=[0.25,0.5,0.75])
```

```
0.25    157.0
0.50    163.0
0.75    170.0
```

# Measures of Dispersion

Measures for the dispersion (spread) in the data include the sample range, $\max_i x_i - \min_i x_i$, the sample variance $s^2$, and the sample standard deviation $s = \sqrt{s^2}$.

```
nutri['height'].max() - nutri['height'].min()
```

```
48.0
```

```
round(nutri['height'].var(), 2)  # round to two decimal places
```

```
81.06
```

```
round(nutri['height'].std(), 2)
```

```
9.0
```

# describe(): Full Numeric Summary

When applied to a quantitative feature, the `describe` method returns the minimum, maximum, mean, and the three quartiles. For example, the `'height'` feature in the nutri data has the following summary statistics.

```
nutri['height'].describe()
```

```
count    226.000000
mean     163.960177
std        9.003368
min      140.000000
25%      157.000000
50%      163.000000
75%      170.000000
max      188.000000
Name: height, dtype: float64
```

# Visualizing Data

Here, we describe various methods for visualizing data.

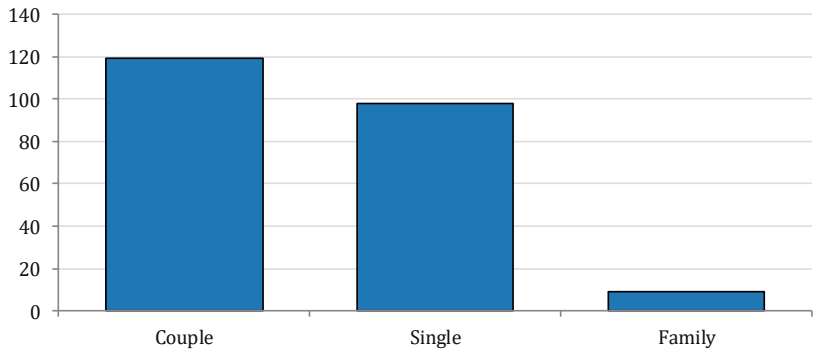
It is assumed that **matplotlib.pyplot**, **pandas**, and **numpy**,
have been imported in the Python code as follows.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

# Plotting Qualitative Variables: Barplot

How many elderly people are living by themselves, as a couple, with family, or other?

```
width = 0.35  # the width of the bars
x = [0, 0.8, 1.6]  # the bar positions on x-axis
situation_counts = nutri['situation'].value_counts()
plt.bar(x, situation_counts, width, edgecolor='black')
plt.xticks(x, situation_counts.index)
plt.show()
```



# Plotting Quantitative Variables: Boxplot

How can we summarize the `'age'` feature of the `nutri` data?

```
plt.boxplot(nutri['age'],widths=width,vert=False)
plt.xlabel('age')
plt.show()
```

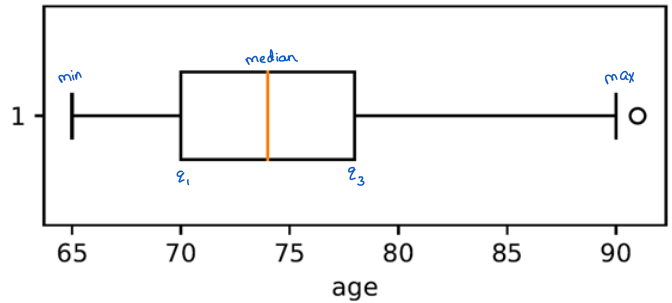


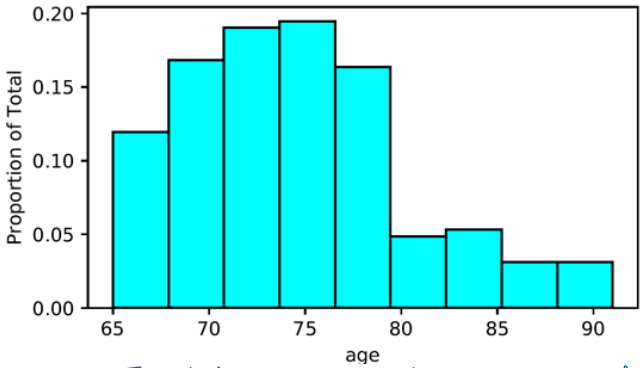

Figure: Boxplot for `'age'`.

# Histogram

What is the age distribution of the elderly?

```
weights = np.ones_like(nutri.age)/nutri.age.count()
plt.hist(nutri.age,bins=9,weights=weights,facecolor='cyan',
         edgecolor='black', linewidth=1)
plt.xlabel('age')
plt.ylabel('Proportion of Total')
plt.show()
```

Histogram represents the distribution of data like the grades of the students (how many students got A+, A, B+, etc.)




This histogram represents how many percentages of ppl are at a certain age (range of ages)

# Empirical Cumulative Distribution Function

```
x = np.sort(nutri.age)
y = np.linspace(0,1,len(nutri.age))
plt.xlabel('age')
plt.ylabel('Fn(x)')
plt.step(x,y)
plt.xlim(x.min(),x.max())
```

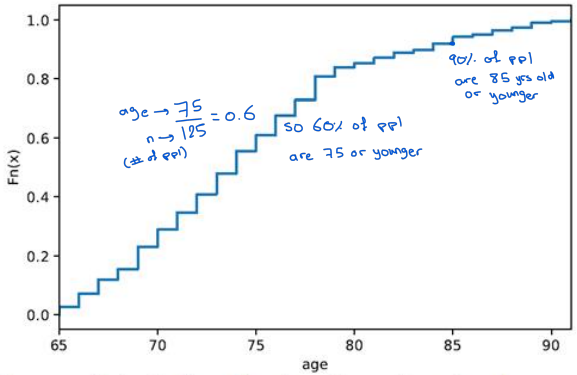



Figure: Plot of the empirical distribution function for the continuous quantitative feature `'age'`.

# Data Visualization in a Bivariate Setting

Comparing barplots for two categorical variables involves introducing subplots.

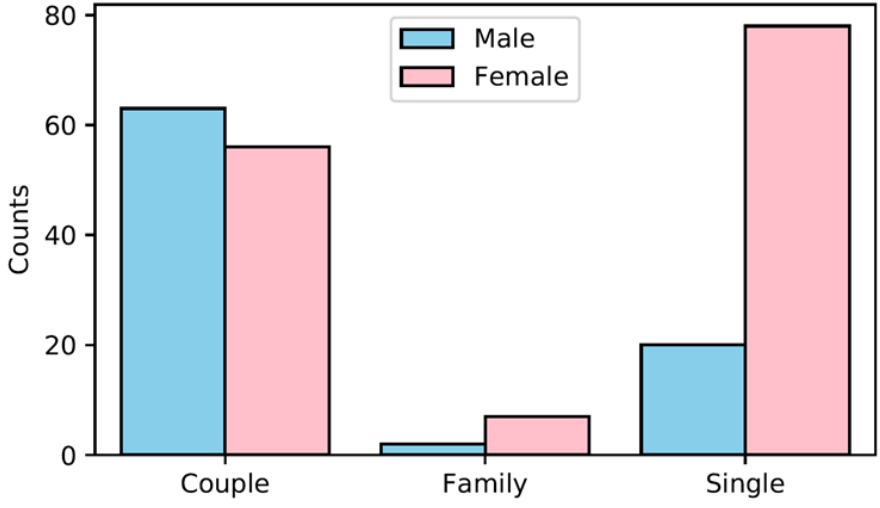



Figure: Barplot for two categorical variables

# Same Plot with Seaborn

```
import seaborn as sns

sns.countplot(x='situation', hue='gender', data=nutri,
              hue_order=['Male', 'Female'],
              palette=['SkyBlue', 'Pink'],
              saturation=1, edgecolor='black')
plt.legend(loc='upper center')
plt.xlabel('')
plt.ylabel('Counts')
```

seaborn's **countplot** produces the same grouped barplot shown on the previous slide directly from the DataFrame, without manually computing counts.

# Plots for Two Quantitative Variables: Scatter Plot

How are height and weight related?

```
plt.scatter(nutri.height, nutri.weight, s=12, marker='o')
plt.xlabel('height')
plt.ylabel('weight')
```

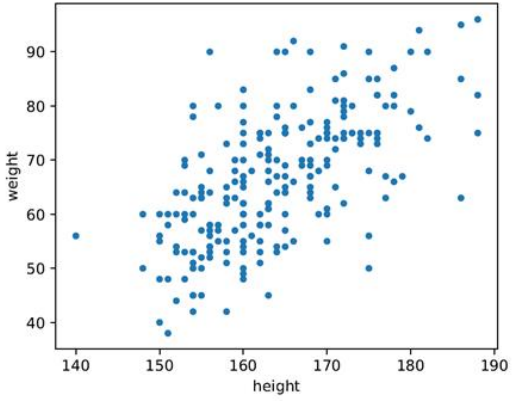



Figure: Scatterplot of `'weight'` against `'height'`.

# Birth Weight vs. Age (Smokers / Non-Smokers)

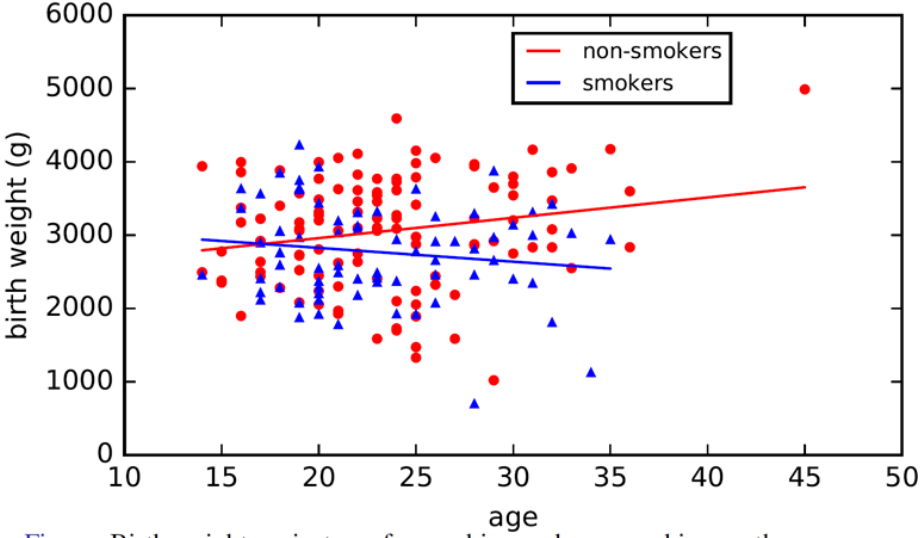



Figure: Birth weight against age for smoking and non-smoking mothers.

# Plots for One Qualitative and One Quantitative Variable

```
males = nutri[nutri.gender == 'Male']
females = nutri[nutri.gender == 'Female']
plt.boxplot([males.coffee,females.coffee],notch=True,widths=(0.5,0.5))
plt.xlabel('gender')
plt.ylabel('coffee')
plt.xticks([1,2],['Male','Female'])
```

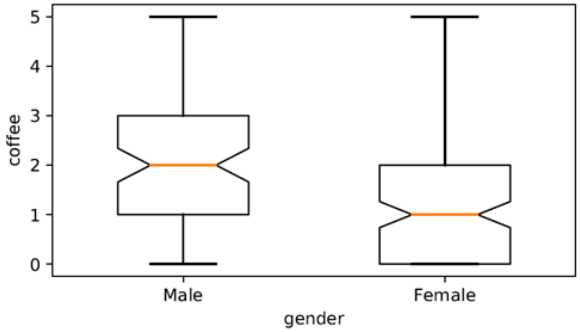



Figure: Boxplots of `'coffee'` as a function of `'gender'`.